계좌 서비스
전체 개선 결과와 실무 재검토

문제 → 수정 이유 → 동작 변화 → 검증 근거

이전 코드 리뷰의 R01-R19를 기준으로 거래 정합성, 보안, 대사, 데이터 이전과 배포 경계를 수정했습니다. 재현 테스트와 통합 검증을 거쳐 원본 프로젝트에 반영한 결과를 정리합니다.

> 재평가: 실무의 기본 통제에 가까워졌습니다. 실자금 운영 승인은 아직 별도입니다. 실제 은행 입금 확인, 업무 한도·역분개·마감 정책과 운영 환경 검증이 추가로 필요합니다.

항목	검증 결과
기존 리뷰	105개 추적 파일 전체 분석 / R01-R19 대응
최종 Java 테스트	35개 클래스 · 134개 통과 · 실패/오류/건너뜀 0
추가 산출물	Flyway V1~V4 · 배포/복구 도구 · 운영/API 문서
검증 환경	Docker MySQL 8.0 / Redis 7 · Java 17 toolchain
수행 범위	로컬 소스 수정 및 검증. 실제 운영 배포·DB 이전 없음

대상 C:/Users/정유현/Desktop/project/account

기준 커밋 7d6f766f7722de5b7ec84e88d4a22537e882ffb2

기존 감사 PDF는 수정 전 근거로 보존합니다. 현재 동작은 이 문서와 README/API_CHANGES/OPERATIONS_RUNBOOK을 기준으로 확인합니다.

돈이 확정되는 경계를 정리했습니다

가장 중요한 변화는 "거래는 성공했는데 중복 방지 기록은 실패한 상태"를 없애는 것입니다. 아래 2~4 단계는 하나의 MySQL 트랜잭션에서 함께 확정되거나 함께 취소됩니다.

1. 인증과 입력 검증
로그인 주체 · 두 자리 금액 · 키 형식 확인

↓

2. 사용자별 키 선점 또는 기존 응답 확인
사용자+키 유일 제약 · 작업/계좌/금액 지문 대조

↓

3. 소유권 확인과 잔액·원장 변경
조건부 UPDATE · 이체의 일관된 갱신 순서 · 감사 ID 저장

↓

4. 응답 저장과 커밋
키 완료 기록과 거래가 같은 트랜잭션에서 확정

↓

5. 응답과 관측
커밋 후 성공 로그 · 재전송은 같은 거래 응답 재생

실패가 발생하면

완료 저장·원장 기록·응답 직렬화가 실패하면 돈도 반영되지 않습니다. DB의 일시적 충돌은 새 트랜잭션에서 최대 5회 시도하고, 한도를 넘으면 C010을 반환합니다. 실패한 트랜잭션 안에서 재생하지 않습니다.

운영에서 자금을 만드는 경계

모의 입금은 prod에서 차단됩니다. 실제 외부 입금 증빙과 승인 절차를 구현한 것으로 간주하지 않습니다. 새로운 업무 기능을 추측해 붙이지 않고 안전한 기본값으로 경계를 분리했습니다.

기존 지적 사항 대응표 1/2

각 ID는 기존 감사 보고서와 같습니다. "구현·경계 검증"은 코드 검증을 완료했으나 실제 인프라나 업무 조건의 증거가 추가로 필요한 항목입니다.

ID	주제	판정
R01	거래와 요청 결과의 원자성	수정·회귀 검증
R02	사용자와 요청 내용에 묶인 키	수정·회귀 검증
R03	금액 단위와 잔액 상한	수정·회귀 검증
R04	모의 입금의 운영 차단	구현·경계 검증
R05	민감값 없는 오류 로그	수정·회귀 검증
R06	전달 헤더와 로그인 제한	구현·경계 검증
R07	운영 Redis 연결과 제한 정책	구현·경계 검증
R08	검증 가능한 DB 변경	구현·경계 검증
R09	공개 TLS와 네트워크 경계	구현·경계 검증
R10	커밋된 거래의 감사 연결	수정·회귀 검증

> 코드 수정 완료와 실제 운영 승인은 구분합니다. TLS 인증서·최소 권한·알림 전달·백업 복원·실사용 부하를 로컬 테스트만으로 승인하지 않습니다.

상세 페이지 읽는 방법

각 항목의 "이전 문제"는 원인과 영향을, "수정"은 현재 동작을, "검증과 한계"는 실제 확인한 결과와 남은 조건을 설명합니다. 파일 경로와 줄 번호는 이번 결과물 기준입니다.

기존 지적 사항 대응표 2/2

각 ID는 기존 감사 보고서와 같습니다. "구현·경계 검증"은 코드 검증을 완료했으나 실제 인프라나 업무 조건의 증거가 추가로 필요한 항목입니다.

ID	주제	판정
R11	대사의 의미와 읽기 일관성	범위 명확화·구현
R12	불일치 증거와 알림	구현·경계 검증
R13	실행 누락과 중단 복구	구현·경계 검증
R14	동일 커밋의 테스트와 배포	구현·경계 검증
R15	이미지 불변성과 배포 순서	구현·경계 검증
R16	실제 요청 경로의 상태 점검	구현·경계 검증
R17	실패를 놓치지 않는 테스트	수정·회귀 검증
R18	키 입력과 제약 오류 분류	수정·회귀 검증
R19	안정적인 이력 순서와 커서	수정·회귀 검증

> 코드 수정 완료와 실제 운영 승인은 구분합니다. TLS 인증서·최소 권한·알림 전달·백업 복원·실사용 부하를 로컬 테스트만으로 승인하지 않습니다.

상세 페이지 읽는 방법

각 항목의 "이전 문제"는 원인과 영향을, "수정"은 현재 동작을, "검증과 한계"는 실제 확인한 결과와 남은 조건을 설명합니다. 파일 경로와 줄 번호는 이번 결과물 기준입니다.

문제와 수정 내용 01

R01 거래와 요청 결과의 원자성

구분	설명
이전 문제	완료 기록 저장 실패 후 재시도하면 돈이 두 번 반영될 수 있었습니다.
수정	키 선점, 잔액 변경, 원장 기록, 응답 완료를 한 MySQL 트랜잭션에 묶었습니다. 재시도는 커밋을 포함한 전체 경계를 다시 시작합니다.
검증과 한계	완료 UPDATE·이력 INSERT·응답 직렬화 실패 시 잔액/원장/키 모두 롤백. 같은 키 재시도는 1회 반영.

코드 src/main/java/com/trading/account/domain/idempotency/IdempotencyService.java:47

R02 사용자와 요청 내용에 묶인 키

구분	설명
이전 문제	전역 키만 같으면 타 사용자 또는 다른 작업의 과거 응답을 받을 수 있었습니다.
수정	사용자+키를 유일하게 저장하고 작업·계좌·정규화 금액 지문을 비교합니다. 수취인 변경도 거절합니다.
검증과 한계	타 사용자 소유 계좌 접근 거절, 다른 금액/작업/수취인 C017, 동등한 금액 표현은 같은 거래 ID.

코드 src/main/java/com/trading/account/domain/transaction/TransactionController.java:45

문제와 수정 내용 02

R03 금액 단위와 잔액 상한

구분	설명
이전 문제	0.015 이체가 DB에서 서로 다르게 반올림되어 총액을 바꾸었습니다.
수정	DTO와 서비스에 공통 금액 검증을 적용하고 잔액 상한을 조건부 UPDATE에 넣었습니다.
검증과 한계	과도한 소수·음수·0·범위 초과 거절. 수취 계좌 상한 초과 시 출금도 롤백.

코드 src/main/java/com/trading/account/domain/account/MoneyAmount.java:14

R04 모의 입금의 운영 차단

구분	설명
이전 문제	계좌 소유자가 입금 API로 잔액을 임의 생성할 수 있었습니다.
수정	기본 비활성화하고 prod에서 항상 차단합니다. dev에서 명시적으로 켠 경우에만 학습용 입금을 허용합니다.
검증과 한계	실제 HTTP 운영 프로필에서 인증한 소유자도 403/C019, 잔액 0 유지. 실제 입금 확인 연동은 별도 범위.

코드 src/main/java/com/trading/account/domain/transaction/TransactionService.java:34

문제와 수정 내용 03

R05 민감값 없는 오류 로그

구분	설명
이전 문제	검증 오류와 DB 오류 원문에 비밀번호·이메일·요청 값이 남을 수 있었습니다.
수정	안전한 코드·예외 종류만 기록합니다. SQL/bind와 SqlExceptionHelper 원문 로그를 차단했습니다.
검증과 한계	잘못된 DTO/JWT 입력과 실제 MySQL 중복 INSERT의 루트 로그·예외 스택에서 민감값 미노출 확인.

코드 src/main/java/com/trading/account/common/exception/GlobalExceptionHandler.java:19

R06 전달 헤더와 로그인 제한

구분	설명
이전 문제	클라이언트가 보낸 X-Forwarded-For로 제한 대상을 바꾸거나 Redis 장애를 우회할 수 있었습니다.
수정	연결 remoteAddr만 사용하고 전달 헤더 처리를 끕니다. Redis 장애는 503으로 거절합니다.
검증과 한계	위조 헤더로 키가 바뀌지 않음, 장애 503. 프록시/NAT 고객별 제한은 경계 장비 정책 검증 필요.

코드 src/main/java/com/trading/account/common/ratelimit/RateLimitFilter.java:73

문제와 수정 내용 04

R07 운영 Redis 연결과 제한 정책

구분	설명
이전 문제	제공된 배포에 필요한 Redis 설정이 없었고 제한 장애를 허용했습니다.
수정	호스트·비밀번호·TLS를 환경에서 주입합니다. Redis TIME과 정확한 버킷 TTL을 사용합니다.
검증과 한계	실제 Redis 50개 동시 요청 중 허용 수 5 확인, refill/TTL 및 잘못된 설정 거절. 실 환경 인증/TLS는 별도 확인.

코드 src/main/java/com/trading/account/common/ratelimit/TokenBucketRateLimiter.java:21

R08 검증 가능한 DB 변경

구분	설명
이전 문제	운영 자동 DDL로 변경 이력과 사전 검증을 통제하기 어려웠습니다.
수정	Flyway V1~V4, prod validate, 별도 마이그레이션 명령과 계정을 도입했습니다. 자동 baseline/clean은 금지합니다.
검증과 한계	빈 DB 설치·재실행·명시 baseline 이전, 잔액/이력/키 보존 및 PAD/NO PAD 문자 규칙을 검증.

코드 src/main/java/com/trading/account/common/config/DatabaseMigration.java:28

문제와 수정 내용 05

R09 공개 TLS와 네트워크 경계

구분	설명
이전 문제	제공된 Ingress에 외부 TLS와 관리 포트 접근 경계가 충분히 드러나지 않았습니다.
수정	websecure+TLS Secret, /api 라우팅, 8080/9090 NetworkPolicy를 제공합니다.
검증과 한계	YAML·템플릿 검증 완료. 인증서, Traefik 라벨, CNI 정책 적용과 외부 차단은 실제 클러스터 확인 필요.

코드 k8s/ingress.yaml:8

R10 커밋된 거래의 감사 연결

구분	설명
이전 문제	커밋 전 성공 로그와 식별자 부족으로 실제 확정 거래를 추적하기 어려웠습니다.
수정	원장에 actor/idempotency 참조를 저장하고 응답에 거래 ID를 추가합니다. 성공 로그는 afterCommit에서만 남깁니다.
검증과 한계	롤백된 거래의 성공 로그 0건. 확정 원장·요청·수행자 ID 연결 확인. 별도 불변 감사 저장소는 추가 운영 범위.

코드 src/main/java/com/trading/account/domain/transaction/TransactionService.java:146

문제와 수정 내용 06

R11 대사의 의미와 읽기 일관성

구분	설명
이전 문제	현재 데이터를 조회하면서 일별 마감처럼 해석할 여지가 있었습니다.
수정	CURRENT_LEDGER_CONSISTENCY로 의미를 명확히 하고 청크별 REPEATABLE_READ와 관측 시각을 적용합니다.
검증과 한계	계좌 조회 뒤 입금이 커밋되어도 같은 청크의 원장 합계와 비교해 거짓 불일치를 만들지 않음. 과거 일마감은 구현 범위 밖.

코드 src/main/java/com/trading/account/domain/reconciliation/BalanceReconciliationJobConfig.java:64

R12 불일치 증거와 알림

구분	설명
이전 문제	대사 불일치가 로그에만 남아 조사·해결 상태를 관리하기 어려웠습니다.
수정	실행/불일치 테이블, 원본 증거 보존 upsert, 상태와 DB 기반 지표 및 경보를 추가합니다.
검증과 한계	재생해도 기존 관측 금액 유지, Prometheus에 실제 지표 출력 확인. 알림 수신 경로는 운영 확인 필요.

코드 src/main/java/com/trading/account/domain/reconciliation/ReconciliationResults.java:41

문제와 수정 내용 07

R13 실행 누락과 중단 복구

구분	설명
이전 문제	예약 시각의 앱 중단으로 대사가 누락되거나 STARTED가 고착될 수 있었습니다.
수정	01시 이후 반복 확인과 같은 날짜 실행 식별자를 사용합니다. 고착 상태는 전체 실행기 정지 후 지정 실행만 복구합니다.
검증과 한계	늦은 기동·FAILED 재시도·완료 중복 방지·살아 있는 STARTED 보호·실제 복구 SQL 검증. 자동 강제 복구는 하지 않음.

코드 src/main/java/com/trading/account/domain/reconciliation/BalanceReconciliationScheduler.java:44

R14 동일 커밋의 테스트와 배포

구분	설명
이전 문제	CI 실패와 배포의 선후 관계를 보장하기 어려웠습니다.
수정	CD가 재사용 CI 성공에 의존하고 동일 Git SHA를 빌드합니다. 운영 배포는 점검 창을 명시한 수동 실행만 허용합니다.
검증과 한계	워크플로 의존 관계·구문 검토. GitHub 보호 규칙과 실제 실행 이력은 미검증.

코드 .github/workflows/cd.yml:24

문제와 수정 내용 08

R15 이미지 불변성과 배포 순서

구분	설명
이전 문제	latest 적용과 겹친 배포로 어떤 코드가 실행되는지 불명확해질 수 있었습니다.
수정	SHA 빌드에서 digest를 확인하고 최종 매니페스트를 한 번 적용합니다. 워크플로 직렬화와 최신 master 확인을 추가했습니다.
검증과 한계	mutable 태그·잘못된 호스트 입력 거절, 정상 digest 렌더링. 운영 rollout 자체는 실행하지 않음.

코드 scripts/render_manifests.py:9

R16 실제 요청 경로의 상태 점검

구분	설명
이전 문제	관리 포트만 정상이면 주 API 프로세스 문제를 놓칠 수 있었습니다.
수정	주 포트 /livez·/readyz를 사용합니다. readiness에 DB/Redis를 포함하고 liveness에는 외부 의존성을 넣지 않습니다.
검증과 한계	실제 HTTP 프로브 200/세부정보 비노출, 기타 관리·Swagger 경로 익명 401. 실제 파드 장애 전환은 별도 검증.

코드 k8s/deployment.yaml:73

문제와 수정 내용 09

R17 실패를 놓치지 않는 테스트

구분	설명
이전 문제	총액만 검사하면 거래가 전부 실패해도 동시성 테스트가 통과할 수 있었습니다.
수정	성공 건수와 이력 수를 함께 확인하고 모든 DB 테스트를 MySQL/Flyway로 통일했습니다. H2를 제거했습니다.
검증과 한계	양방향 이체 성공 수·원장 건수·잔액 합계, 동일 키 50개 동시 요청 1회 원장 반영 확인.

코드 src/test/java/com/trading/account/domain/transaction/TransactionConcurrencyTest.java:27

R18 키 입력과 제약 오류 분류

구분	설명
이전 문제	키 길이·문자 계약과 중복 오류 구분이 불명확했습니다.
수정	키를 1~100자의 제한된 문자로 검증하고 명명한 MySQL 유일 제약 위반만 재생 경로로 처리합니다.
검증과 한계	다른 무결성 오류는 재생하지 않음. legacy 키의 대소문자·악센트·후행 공백 비교 규칙 보존.

코드 src/main/java/com/trading/account/domain/idempotency/IdempotencyService.java:79

문제와 수정 내용 10

R19 안정적인 이력 순서와 커서

구분	설명
이전 문제	같은 시각 이력의 순서가 흔들리고 무제한/임의 정렬이 조회 계약을 약하게 했습니다.
수정	유일 ID 내림차순과 1~100 크기를 고정하고 beforeId 커서 API를 추가했습니다.
검증과 한계	동일 timestamp 순서 고정, 새 거래 추가 중 다음 커서 페이지 중복 없음. 전역 불변 스냅샷 보장은 아님.

코드 src/main/java/com/trading/account/domain/transaction/TransactionService.java:109

통합 리뷰에서 추가로 막은 경로

DB 마이그레이션 URL의 잘못된 sslmode/SSLMODE, 중복·인코딩된 TLS 옵션, per-host 설정 우회를 검사했습니다. 드라이버가 인식하는 정확한 sslMode=VERIFY_IDENTITY 계약을 검증합니다.

Hibernate는 예외 처리기에 도착하기 전에 중복 이메일 원문을 ERROR로 남길 수 있었습니다. SqlExceptionHelper 자체를 끄고 실제 MySQL 중복 INSERT 로그로 확인했습니다.

테스트가 증명하는 내용

검증 묶음	확인한 결과
최종 전체 실행	134개 / 실패 0 / 오류 0 / 건너뜀 0. 전체 테스트와 bootJar 성공.
거래 장애와 경쟁	키 완료/원장/직렬화 실패 시 전부 롤백. 동일 키 50개 동시 요청은 원장 1회 반영. 이체 성공 건수와 이력 수까지 확인.
금액과 권한	0.015 거절, 잔액 상한, 사용자·작업·금액·수취인 변경 검증. prod 실제 HTTP 모의 입금 차단.
보안과 제한	JWT/입력 경계, 헤더 위조, Redis 장애 차단, 실제 Redis 동시 요청·TTL. 민감값 로그 검사.
이전과 대사	빈 DB·기존 baseline·문자 비교 규칙·이전 데이터 보존. 대사 증거/복구 SQL/지표 실출력.
배포 산출물	렌더러 3개 테스트, YAML 15개 파일 검증. Docker 빌드와 비루트/읽기 전용 CLI 실행 확인.

실패하는 테스트를 먼저 확인한 사례

원래 거래 경계 회귀 12개 중 11개, 직렬화/성공 로그 2개, legacy 별칭 1개가 수정 전에 실패했습니다. 통합 리뷰에서도 TLS 옵션·DB 로그 2개와 NO PAD 이전 1개를 실패 재현한 뒤 수정했습니다. 개별 실행 수를 최종 전체 테스트 수에 더하지 않습니다.

검증하지 않은 범위

실클러스터 배포, 네트워크 단절/프로세스 강제 종료 전체 조합, 장시간 실부하, 백업/PITR 복원, 외부 입금 정산, 취약점/SBOM 전수 검사는 수행하지 않았습니다. 실행 시간은 성능 목표의 증거로 사용하지 않습니다.

기존 데이터를 안전하게 넘기는 순서

버전	책임
V1	기존 회원·계좌·거래 이력·멱등성 테이블
V2	Spring Batch 5.2 메타데이터와 시퀀스
V3	사용자별 키/지문·legacy 비교 규칙·거래 감사 연결
V4	대사 실행과 불일치 증거

기존 DB에는 계획된 중단이 필요합니다

1. 백업을 만들고 복원 성공을 확인합니다. 실제 스키마와 V1, 기존 Batch 구조를 비교합니다.

2. 모든 구버전 앱과 다른 대사 실행기를 중지합니다. 이번 버전은 구버전과 동시 쓰기를 지원하지 않습니다.

3. 별도 DDL 계정으로 명시적 baseline(버전 1) → migrate → validate를 실행합니다. 빈 DB에는 baseline 없이 migrate합니다.

4. 계좌 수·잔액 합계·이력 수·기존 키 수를 대조한 후 신버전과 readiness를 확인합니다.

이전 키는 삭제하거나 추측하지 않습니다

소유자와 원래 요청을 복원할 수 없는 키는 actor 0 / LEGACY로 격리합니다. legacy_key에는 기존 문자 비교 규칙을 보존합니다. 새 주 키는 utf8mb4_0900_bin을 사용해 과거의 후행 공백 구분도 잃지 않습니다.

> DDL 중간 실패 시 자동으로 구버전을 재시작하지 않습니다. MySQL DDL 일부는 이미 반영됐을 수 있습니다. 스키마·Flyway 이력을 조사하고 복원 또는 검토한 전진 수정으로 대응합니다.

명령과 환경변수: docs/OPERATIONS_RUNBOOK.md. 이번 작업은 실제 사용자 DB에 마이그레이션을 실행하지 않았습니다.

검증한 버전만 배포하는 구조

1. 동일 SHA의 재사용 CI
전체 Java 테스트와 매니페스트 렌더러 테스트

↓

2. SHA 이미지 빌드 및 digest 확인
테스트 성공에 의존 · mutable latest 배포 제거

↓

3. 운영 사전 점검
수동 maintenance_window · 현재 master · Secret/HPA/매니페스트 확인

↓

4. 계획된 중단과 마이그레이션
기존 파드 0개 확인 · 별도 계정 Job 성공 필요

↓

5. 최종 digest 적용
주 포트 readiness가 통과해야 rollout 완료

접근 경계

Ingress는 TLS의 /api만 공개합니다. 네트워크 정책은 앱 8080에 Traefik, 관리 9090에 monitoring namespace만 허용합니다. 앱 컨테이너는 비루트 사용자, 읽기 전용 루트 파일시스템과 임시 쓰기 볼륨을 사용합니다.

환경에서 받아야 할 증거

GitHub environment 승인 규칙, ECR 권한, 인증서 유효성, Traefik 라벨, CNI NetworkPolicy 동작과 실제 rollout은 운영 환경에서 확인해야 합니다. 로컬 YAML 파싱을 클러스터 배포 성공으로 표현하지 않습니다.

대사와 장애 대응의 책임을 명확히 했습니다

현재 정합성 점검

현재 잔액과 현재 원장 합계를 청크 단위의 같은 스냅샷에서 비교합니다. 각 실행과 불일치 관측 시각을 남깁니다. 모든 계좌가 하나의 전역 시점으로 고정되거나 과거 영업일의 마감 잔액이 재현되는 기능은 아닙니다.

증거/지표	의미
reconciliation_run	실행 기준·일정 날짜·상태·완료 시각·불일치 수
reconciliation_mismatch	실제/기대 잔액·관측 시각·조사 상태. 재생해도 원본 증거 유지
reconciliation_mismatches	최근 완료 실행의 불일치 수
reconciliation_mismatches_recorded	누적 보관 불일치 행 수. _total과 다른 이름
last_completed / last_success	완료 실행 / 불일치 없는 실행의 UTC epoch 시각
runs_failed / launch_failures_total	DB 실패 이력 / 프로세스 실행 요청 실패

멈춘 실행은 안전한 점검 창에서만 복구

시각이 오래됐다는 이유만으로 STARTED를 강제 재시작하지 않습니다. 복구 스크립트는 앱 replicas=0, 잔존 파드 없음, 연결된 HPA 없음과 다른 실행기 중단 확인을 요구합니다. 지정한 Batch 실행만 FAILED로 바꾸고 기존 조사 증거를 보존합니다.

복구 SQL은 실제 MySQL에서 실행·재시도·완료 상태 거절을 검증했습니다. 운영자가 중단을 보장해야 하는 방식이며 자동 분산 fencing이나 무중단 장애 전환은 아닙니다.

실무에 얼마나 가까워졌는가

> 판정: 실무 원칙을 적용한 모의 금융 서비스로 개선되었습니다. 실자금 서비스를 바로 운영할 수 있다는 결론은 아닙니다.

영역	현재 수준	다음 승인 조건
거래 정합성	원자적 멱등성·금액 경계·회귀 테스트 확보	실제 규모의 부하·장애 주입, 업무 한도 정의
보안	민감값 로그·인증 입력·제한 장애 경계 개선	키 회전/폐기·계정 정책·의존성 보안 점검
데이터 변경	명시적 버전 이전·구버전 격리·검증	실제 스키마 비교, 최소 권한, 복원 훈련
운영	불일치 보관·지표·중단 복구 절차	알림 전달·담당자 대응·RTO/RPO 입증
배포	동일 SHA 테스트·digest·직렬화·프로브	실클러스터 네트워크/TLS/배포 증거
금융 업무	입금 모의 기능을 운영에서 차단	외부 입금 확인, 계좌 상태, 한도, 승인/역분개, 통화·마감 정책

단순함을 유지한 결정

단일 MySQL과 기존 서비스 계층을 유지했습니다. 이번 수정 때문에 별도 메시지 브로커, 분산 트랜잭션, 범용 정책 프레임워크를 추가하지 않았습니다. 필요한 불변식과 복구 가능성을 현재 규모에서 검증하는 데 집중했습니다.

점수를 부여하지 않은 이유

운영 승인에는 코드 밖의 증거가 필요합니다. 임의의 "금융권 수준 90점"보다 확인한 통제와 미확인 조건을 분리하는 편이 판단에 도움이 됩니다.

프로젝트에서 다시 확인하는 방법

문서	용도
HARDENING_REPORT.md / 이 PDF	문제와 수정, 최종 실행 검증, 실무 재평가
API_CHANGES.md	키 재사용·금액·응답 식별자·커서·오류 계약
OPERATIONS_RUNBOOK.md	실행 환경, 스키마 이전, 릴리스, 복구 절차
CORE / SECURITY / RECONCILIATION 노트	분야별 수정 근거와 상세 테스트
HARDENING_CHANGED_FILES.md	파일별 수정/추가 목록과 역할
HISTORICAL_CONCURRENCY_NOTES.md	개선 전 README 학습 기록 보존

개발자가 확인할 순서

1. Docker를 켜고 gradlew.bat test, gradlew.bat bootJar를 실행합니다.
2. python scripts/test_render_manifests.py로 배포 입력 계약을 확인합니다.
3. dev 환경변수와 모의 입금 플래그를 설정해 새 API 응답과 재시도를 확인합니다.
4. 운영 적용 전에는 Runbook의 백업/중단/이전/검증 순서를 따릅니다.

보존한 것

사용자의 .claude, ignored 로컬 설정과 기존 데이터에는 손대지 않았습니다. 변경 전 파일은 별도 작업 폴더에 백업하고 최초 파일 해시와 대조하여 적용했습니다. Git commit/push와 운영 배포는 수행하지 않았습니다.

이전 감사 문서의 위험 재현 결과는 수정 전 근거입니다. 현재 테스트 결과와 혼동하지 않도록 둘 다 보존합니다.